

Optimization Methods for Machine Learning and Engineering

Lecture 9 – Classification, SVM and the Reproducing Kernel Hilbert Space

Julius Pfrommer

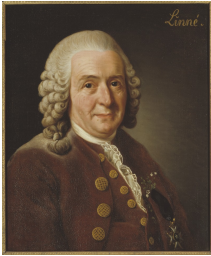


CC BY-SA 4.0

1. Linear Classification
2. Nonlinear Classification with Kernels
3. Reproducing Kernel Hilbert Spaces

Linear Classification

Origins of Classification



Carl von Linné (1707 – 1778)



Systema Naturæ (1758)

Image Source: Wikipedia

Already **Aristotle** in the 4th century BC came up with a taxonomy of animals and plants.

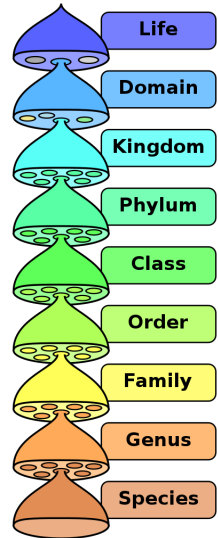
In the 18th century, **Carl von Linné** defined a 5-level hierarchy to classify all plants and animals. It is still in use, albeit in modified form.

von Linné also invented the **binomial naming scheme** for genus and species:

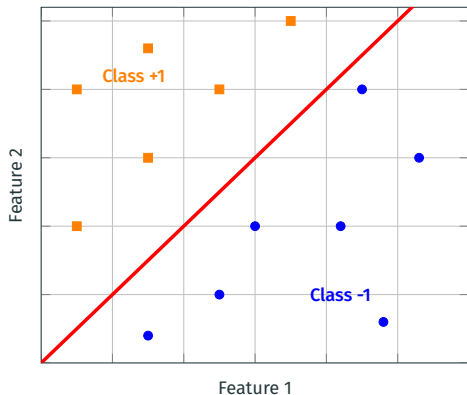
Boa Constrictor
Genus Species Epiteth

Family: Boidae, Order: Squamata, Class: Reptilia,
Phylum: Chordata, Kingdom: Animalia, Domain: Eukarya

There are an estimated 8–8.7 million species on earth. Many of them undocumented.



Linear Classification



Classification by separation with a hyperplane was first described in: Ronald A Fisher. “The use of multiple measurements in taxonomic problems”. In: *Annals of Eugenics* 7.2 (1936), pp. 179–188.

Binary classification is about separating data points $\mathbf{x}$ into either of two **categorical groups (classes)**. Here we consider two classes $\{1, -1\}$.

Let a dataset $D = \{(\mathbf{x}_1, l_1), \dots, (\mathbf{x}_d, l_d)\}$ with **samples** $\mathbf{x}_i$ and their respective **ground-truth label** $l_i \in \{-1, +1\}$.

Linear binary classification constructs a fitting decision boundary by choosing **weights** $\mathbf{w}$ and an **intercept** b .

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

The predicted class $\hat{l}_i$ depends on “which side of the decision boundary” $\mathbf{x}_i$ ends up on:

$$\hat{l}_i = \text{sign}(\mathbf{w}^\top \mathbf{x}_i + b)$$

A label l_i is predicted correctly if $l_i(\mathbf{w}^\top \mathbf{x}_i + b) > 0$.

The Perceptron Algorithm

Shorthand for the misclassified samples:

$$\Gamma_D(\mathbf{w}, b) = \{i \mid l_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0\}$$

```
1: procedure TRAINPERCEPTRON( $D$ )
2:    $\mathbf{w} \leftarrow \mathbf{0}$ 
3:    $b \leftarrow 0$ 
4:   while  $\Gamma_D(\mathbf{w}, b) \neq \emptyset$  do
5:      $i \leftarrow \text{rand}(\Gamma_D(\mathbf{w}, b))$ 
6:      $\mathbf{w} \leftarrow \mathbf{w} + l_i \mathbf{x}_i$ 
7:      $b \leftarrow b + l_i$ 
8:   end while
9:   return  $\mathbf{w}, b$ 
10: end procedure
```

```
1: procedure CLASSIFYLINEAR( $\mathbf{x}, \mathbf{w}, b$ )
2:   return  $\text{sign}(\mathbf{w}^\top \mathbf{x} + b)$ 
3: end procedure
```

The **Perceptron** [Rosenblatt1958] is an early **neural network**. A single-layer Perceptron is a **linear classifier**.

When a **randomly selected training sample** $\mathbf{x}_i$ has the wrong predicted label, the model parameters are “nudged” in the direction of the correct label.

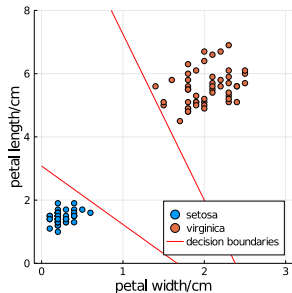
The Perceptron algorithm will **converge within a finite number of steps** if the classes are **linearly separable**.

The classifier result is **invariant to scalar multiplication** of the parameters $(\mathbf{w}, b)$:

$$\text{sign}((c\mathbf{w})^\top \mathbf{x} + cb) = \text{sign}(c(\mathbf{w}^\top \mathbf{x} + b)) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$$

So we can normalize (e.g. so that $\|\mathbf{w}\| = 1$) to avoid numerical issues for many training iterations.

Applying the Perceptron Algorithm on the Iris Data Set



The Iris Data Set is a **collection of data** from three different iris species: IRIS VERSICOLOR, IRIS SETOSA and IRIS VIRGINICA

It includes 50 samples with measurements for petal and sepal length for each species. In 1936, Ronald Fisher used it for publications on **linear discriminant analysis**. Since then, it has become one of the canonical data sets in machine learning.

The Perceptron will find a decision boundary with **zero classification error** on the training data between the setosa and virginica variants, as they are **linearly separable**.

But the Perceptron algorithm is randomized. It will come with **different parameters** (w, b), depending on the **processing order** of the training samples.

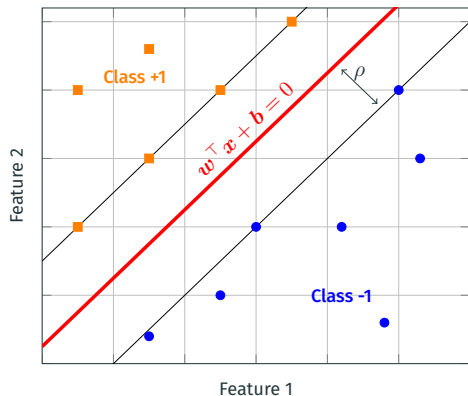


(a) IRIS SETOSA



(b) IRIS VIRGINICA

Support Vector Machine



SVM as an Optimization Problem (Incomplete)

$$\max_{w \in \mathbb{R}^n, b \in \mathbb{R}} \rho$$

$$\text{subject to } l(w^T x + b) > 0, \quad \forall (x, l) \in D$$

The **Support Vector Machine (SVM)** [Boser1992] is called a **maximum-margin classifier**. It tries to maximize the margin ρ between the decision boundary and the samples. This makes the classifier **more robust**.

The SVM is **solved as a convex optimization problem** and does not depend on the order of the training data.

For **hard-margin SVM** variant no sample must end up on the wrong side of the decision boundary. **Soft-margin SVM** make a trade-off between the margin width and the resulting wrong classifications. This is a type of **regularization** that prefers “simpler” decision boundaries.

The left-hand formula is still incomplete, as we haven't yet defined how the margin ρ follows from the model parameters (w, b) and the data D .

SVM as a Convex Optimization Problem

Let $(\mathbf{x}_+, l_+)$ the sample closest to the decision boundary. (Any of the closest samples if not unique.)

The minimum distance projection of $\mathbf{x}_+$ on the decision boundary gives the point $\mathbf{h}$.

Note that the vector $\mathbf{w}$ is orthogonal to the decision boundary.

$$\mathbf{x}_+ = \mathbf{h} + \rho \frac{\mathbf{w}}{\|\mathbf{w}\|_2}$$

Multiply both sides with $\mathbf{w}$:

$$\mathbf{w}^\top \mathbf{x}_+ = \mathbf{w}^\top \left(\mathbf{h} + \rho \frac{\mathbf{w}}{\|\mathbf{w}\|_2} \right)$$

Add b to both sides:

$$\mathbf{w}^\top \mathbf{x}_+ + b = \underbrace{\mathbf{w}^\top \mathbf{h} + b}_0 + \rho \|\mathbf{w}\|_2$$

Due to invariance of the model parameters $(\mathbf{w}, b)$ to scalar multiplication, we can scale so that

$|\mathbf{w}^\top \mathbf{x}_+ + b| = 1$. This gives us the result $\boxed{\rho = 1/\|\mathbf{w}\|_2}$.

The **hard-margin SVM parameters** are the solution of a **convex optimization problem**:

$$\max_{\mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R}} \quad 1/\|\mathbf{w}\|_2$$

$$\text{subject to} \quad l(\mathbf{w}^\top \mathbf{x} + b) \geq 1, \quad \forall (\mathbf{x}, l) \in D$$

$\Leftrightarrow$

$$\min_{\mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R}} \quad \frac{1}{2} \|\mathbf{w}\|_2^2$$

$$\text{subject to} \quad l(\mathbf{w}^\top \mathbf{x} + b) \geq 1, \quad \forall (\mathbf{x}, l) \in D$$

Nonlinear Classification with Kernels

Basis Expansion and Cover's Theorem

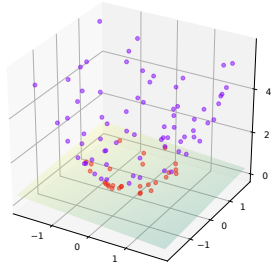
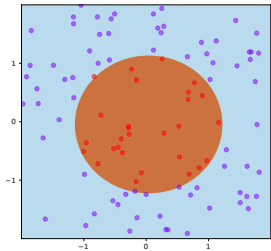


Image Source: Wikipedia

"A complex pattern-classification problem, cast in a high-dimensional space nonlinearly, is more likely to be linearly separable than in a low-dimensional space, provided that the space is not densely populated."

Thomas Cover [Cover1965]

The **nonlinear transformation function** $\Phi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ maps each feature vector $\mathbf{x}$ in dimension n to a higher dimension m .

This is called **basis expansion**. We want the basis expansion to be **injective** to be useful for classification.

Find a linear decision boundary in the expanded basis $\Rightarrow$ Nonlinear Classification

2D Polynomial Basis Expansion

$$\Phi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)^\top$$

Gaussian Basis Expansion

$$\Phi(\mathbf{x}) = (\phi_1(\mathbf{x}), \dots, \phi_m(\mathbf{x}))^\top$$

$$\text{for } \phi_j(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \boldsymbol{\mu}_j\|^2}{2\sigma_j^2}\right), \text{ known } (\boldsymbol{\mu}_j, \sigma_j)$$

The Nonlinear Perceptron and the Curse of Dimensionality of Basis Expansion

Shorthand for the misclassified samples:

$$\Gamma_D(\mathbf{w}, b) = \{i \mid l_i(\mathbf{w}^\top \Phi(\mathbf{x}_i) + b) \leq 0\}$$

```
1: procedure TRAINNONLINEARPERCEPTRON( $D$ )
2:    $\mathbf{w} \leftarrow \mathbf{0}$ 
3:    $b \leftarrow 0$ 
4:   while  $\Gamma_D(\mathbf{w}, b) \neq \emptyset$  do
5:      $i \leftarrow \text{rand}(\Gamma_D(\mathbf{w}, b))$ 
6:      $\mathbf{w} \leftarrow \mathbf{w} + l_i \Phi(\mathbf{x}_i)$ 
7:      $b \leftarrow b + l_i$ 
8:   end while
9:   return  $\mathbf{w}, b$ 
10: end procedure

1: procedure CLASSIFYNONLINEAR( $\mathbf{x}, \mathbf{w}, b$ )
2:   return  $\text{sign}(\mathbf{w}^\top \Phi(\mathbf{x}) + b)$ 
3: end procedure
```

MNIST examples with 28×28 grayscale features each



How many dimensions can we allow for the original and the expanded feature space? In principle there is not limit. However...

Consider the MNIST data set with 7000 samples in 784 dimensions. Even for a simple expansion to the 2nd-degree polynomials for the decision boundary, we would have to expand the features from $\mathbb{R}^{784}$ to $\mathbb{R}^{308504}$.

That is not efficient at all!

The Nonlinear Dual Perceptron

Shorthand for the misclassified samples:

$$\Gamma_D(\alpha, b) = \{i \mid l_i(\langle \sum_j [\alpha_j l_j \Phi(\mathbf{x}_j)] \mid \Phi(\mathbf{x}_i) \rangle + b) \leq 0\}$$

```
1: procedure TRAINNONLINEARDUALPERCEPTRON( $D$ )
2:    $\alpha \leftarrow \mathbf{0}$ 
3:    $b \leftarrow 0$ 
4:   while  $\Gamma_D(\alpha, b) \neq \emptyset$  do
5:      $i \leftarrow \text{rand}(\Gamma_D(\alpha, b))$ 
6:      $\alpha_i \leftarrow \alpha_i + 1$ 
7:      $b \leftarrow b + l_i$ 
8:   end while
9:   return  $\alpha, b$ 
10: end procedure

1: procedure CLASSIFYNONLINEARDUAL( $\mathbf{y}, D, \alpha, b$ )
2:   return  $\text{sign}(\langle \sum_j [\alpha_j l_j \Phi(\mathbf{x}_j)] \mid \Phi(\mathbf{y}) \rangle + b)$ 
3: end procedure
```

We now **generalize to any Hilbert space** and write $\langle \mathbf{x} \mid \mathbf{y} \rangle$ instead of $\mathbf{x}^\top \mathbf{y}$ and $\langle \Phi(\mathbf{x}) \mid \Phi(\mathbf{y}) \rangle$ instead of $\Phi(\mathbf{x})^\top \Phi(\mathbf{y})$.

Use a **count vector** $\alpha \in \mathbb{N}^d$ instead of a weights vector $\mathbf{w} \in \mathbb{R}^m$. The α_i count the **number of times the i th sample $\mathbf{x}_i$ was used** to update the weights.

The original weights vector $\mathbf{w}$ can be reconstructed as

$$\mathbf{w} = \sum_{j=1}^d \alpha_j l_j \Phi(\mathbf{x}_j).$$

This is not very efficient yet... Wait for it!

Note that the Dual Perceptron algorithm has nothing to do with the Lagrangian Dual. The “dual” term is heavily overloaded in mathematics / computer science.

The Kernel Trick: A Shortcut for Basis Expansion + Inner Product

Recall the **polynomial basis expansion**: $\Phi(x) = (x_1^n, \dots, x_n^n, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \dots, \sqrt{2}x_{n-1}x_n)^\top$

Let the **kernel function** $k(x, y) = \langle \Phi(x) \mid \Phi(y) \rangle$. Expanding this kernel with the Euclidean inner product:

Two-Dimensional Features

$$\begin{aligned}k(\mathbf{a}, \mathbf{b}) &= \langle \Phi\left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}\right) \mid \Phi\left(\begin{pmatrix} b_1 \\ b_2 \end{pmatrix}\right) \rangle \\&= \begin{pmatrix} a_1^2 \\ \sqrt{2}a_1a_2 \\ a_2^2 \end{pmatrix}^\top \begin{pmatrix} b_1^2 \\ \sqrt{2}b_1b_2 \\ b_2^2 \end{pmatrix} \\&= a_1^2b_1^2 + 2a_1b_1a_2b_2 + a_2^2b_2^2 \\&= (a_1b_1 + a_2b_2)^2 = (\mathbf{a}^\top \mathbf{b})^2\end{aligned}$$

Higher-Dimensional Features (Multinomial Theorem)

$$\begin{aligned}k(\mathbf{x}, \mathbf{y}) &= \sum_{i=1}^n [x_i^2 y_i^2] + \\&\quad \sum_{i=2}^n \sum_{j=1}^{n-1} [(\sqrt{2}x_i x_j)(\sqrt{2}y_i y_j)] \\&= \sum_{i=1}^n [x_i y_i]^2 = \boxed{(\mathbf{x}^\top \mathbf{y})^2}\end{aligned}$$

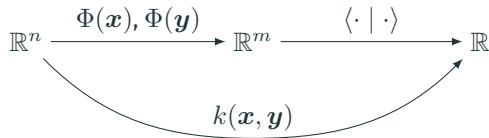
In higher dimensions, it is **much faster to compute** $k(x, y) = (\mathbf{x}^\top \mathbf{y})^2$ than to explicitly expand the basis!

There are other kernels that imply even a **infinite-dimensional basis expansion**. We can nevertheless evaluate the kernel without explicitly computing the expanded feature vector.

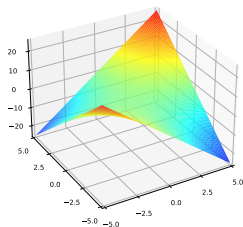
Some Example Kernels

Using a **symmetric kernel function** k is a computationally efficient shortcut when we can avoid to calculate the expansion to $\Phi(\mathbf{x})$ and $\Phi(\mathbf{y})$.

The last section of this lecture on **Reproducing Kernel Hilbert Spaces** (RKHS) discusses necessary conditions for a function to be a valid kernel.

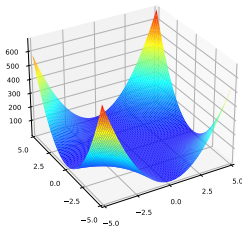


Linear Kernel



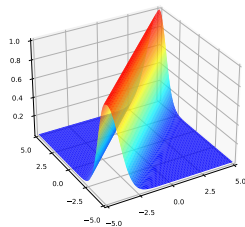
$$k(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y} + c$$

Polynomial Kernel



$$k(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^\top \mathbf{y} + c)^d$$

RBF Kernel



$$k(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{|\mathbf{x} - \mathbf{y}|^2}{2\sigma^2}\right)$$

Other popular kernels are the **Mahalanobis Kernel**, **Cauchy Kernel**, etc.

The Kernel Perceptron

Shorthand for the misclassified samples:

$$\Gamma_D(\alpha, b) = \{i \mid l_i(\sum_j [\alpha_j l_j k(\mathbf{x}_j, \mathbf{x}_i)] + b) \leq 0\}$$

```
1: procedure TRAINKERNELPERCEPTRON( $D$ )
2:    $\alpha \leftarrow \mathbf{0}$ 
3:    $b \leftarrow 0$ 
4:   while  $\Gamma_D(\alpha, b) \neq \emptyset$  do
5:      $i \leftarrow \text{rand}(\Gamma_D(\alpha, b))$ 
6:      $\alpha_i \leftarrow \alpha_i + 1$ 
7:      $b \leftarrow b + l_i$ 
8:   end while
9:   return  $\alpha, b$ 
10: end procedure
```

```
1: procedure CLASSIFYKERNEL( $\mathbf{y}, D, \alpha, b$ )
2:   return  $\text{sign}(\sum_j [\alpha_j l_j k(\mathbf{x}_j, \mathbf{y})] + b)$ 
3: end procedure
```

Image Source: <http://dataphiles.blogspot.com>

The Kernel Perceptron has the **identical behavior to the primal nonlinear formulation**, without the basis expansion.

$$\begin{aligned}\langle \mathbf{w} \mid \Phi(\mathbf{y}) \rangle &= \langle \sum_j [\alpha_j l_j \Phi(\mathbf{x}_j)] \mid \Phi(\mathbf{y}) \rangle \\ &= \sum_j \alpha_j l_j \langle \Phi(\mathbf{x}_j) \mid \Phi(\mathbf{y}) \rangle \\ &= \sum_j \alpha_j l_j k(\mathbf{x}_j, \mathbf{y})\end{aligned}$$

Victory! We got rid of the basis expansion. What now?



The Dual of the SVM

Primal SVM Problem

$$\begin{aligned} \min_{\mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R}} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 \\ \text{subject to} \quad & l_i(\langle \mathbf{w} \mid \mathbf{x}_i \rangle + b) \geq 1, \quad i = 1, \dots, d \end{aligned}$$

Lagrangian of the Primal SVM Problem

$$\begin{aligned} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\mu}) &= \frac{1}{2} \|\mathbf{w}\|_2^2 + \\ &\quad \sum_{i=1}^d [\mu_i (1 - l_i(\langle \mathbf{w} \mid \mathbf{x}_i \rangle + b))] \\ \text{subject to} \quad & \mu_i > 0 \end{aligned}$$

The inner-product is now used also for the SVM. But we **cannot apply the kernel-trick to $\langle \mathbf{w} \mid \mathbf{x}_i \rangle$** . It only works for two data-vectors $\mathbf{x}_i, \mathbf{x}_j$.

For shorter notation we write $\mathbf{l}$ for the vector of all labels l_i . We know that in the saddle-point

$$\frac{\partial}{\partial b} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\mu}) = \boldsymbol{\mu}^\top \mathbf{l} = 0 \quad \text{and} \quad \frac{\partial}{\partial \mathbf{w}} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\mu}) = \mathbf{w} - \sum_j \mu_j l_j \mathbf{x}_j = \mathbf{0}.$$

Plug this in. After transforming, we find that **in the saddle-point, $\mathcal{L}$ no longer depends on $\mathbf{w}$ and b !**

$$\mathcal{L}(\mathbf{w}^*, b^*, \boldsymbol{\mu}^*) = \sum_i \mu_i - \frac{1}{2} \sum_{i,j} \mu_i \mu_j l_i l_j \langle \mathbf{x}_i \mid \mathbf{x}_j \rangle$$

The **Lagrangian dual** of SVM training: $q(\boldsymbol{\mu}) = \inf_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\mu}) = \sum_i \mu_i - \frac{1}{2} \sum_{i,j} \mu_i \mu_j l_i l_j \langle \mathbf{x}_i \mid \mathbf{x}_j \rangle$

Solving and Kernelizing the Support-Vector Machine

Solving the dual of the hard-margin SVM:

$$\begin{aligned} \max_{\boldsymbol{\mu} \in \mathbb{R}^d} \quad & \sum_i^d \mu_i - \frac{1}{2} \sum_{i,j=1}^d \mu_i \mu_j l_i l_j \langle \mathbf{x}_i | \mathbf{x}_j \rangle \\ \text{subject to} \quad & \boldsymbol{\mu} \geq 0, \quad \boldsymbol{\mu}^\top \mathbf{l} = 0 \end{aligned}$$

For a symmetric matrix $\mathbf{K} = \begin{pmatrix} l_1 l_1 \langle \mathbf{x}_1 | \mathbf{x}_1 \rangle & \cdots & l_1 l_d \langle \mathbf{x}_1 | \mathbf{x}_d \rangle \\ \vdots & \ddots & \vdots \\ l_d l_1 \langle \mathbf{x}_d | \mathbf{x}_1 \rangle & \cdots & l_d l_d \langle \mathbf{x}_d | \mathbf{x}_d \rangle \end{pmatrix}$ the objective function is $\mathbf{1}^\top \boldsymbol{\mu} - \frac{1}{2} \boldsymbol{\mu}^\top \mathbf{K} \boldsymbol{\mu}$.

Let the set of “**active constraints**” $A = \{i \mid \mu_i > 0\}$ which are the actual **support vectors**. Typically A is very small. A new sample $\mathbf{y}$ used to be classified as $\text{sign}(\langle \mathbf{w} | \mathbf{y} \rangle + b)$. Now **directly from $\boldsymbol{\mu}$** :

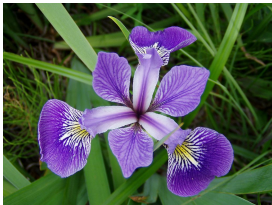
$$\langle \mathbf{w} | \mathbf{y} \rangle + b = \underbrace{\sum_{a \in A} [\mu_a l_a \langle \mathbf{x}_a | \mathbf{y} \rangle]}_{\langle \mathbf{w} | \mathbf{y} \rangle} + \underbrace{\frac{1}{|A|} \sum_{a \in A} [l_a - \sum_i l_i \mu_i \langle \mathbf{x}_i | \mathbf{x}_a \rangle]}_b$$

The dual version of the SVM is **easy to kernelize**. Just replace every $\langle \mathbf{x}_i | \mathbf{x}_j \rangle$ with $k(\mathbf{x}_i, \mathbf{x}_j)$.

Once trained, the dual SVM is comparatively **efficient**. Because of the **kernel trick** and since only a **few support vectors** from A are needed for the classification of a test sample.

Example: Kernel SVM with an RBF Kernel

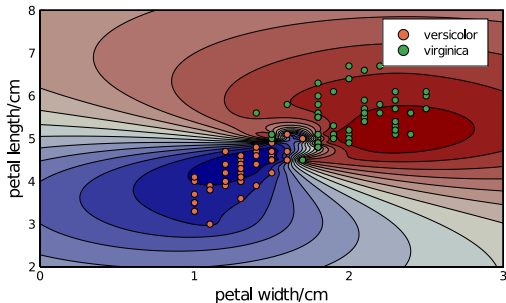
IRIS VERSICOLOR



IRIS VIRGINICA



Image Source: Wikipedia



For two very similar types of iris the classes seem to overlap!

The Kernel-SVM with an RBF Kernel ($\sigma = 1$) classifies all training samples correctly. But complicated decision boundaries are very prone to overfitting.

With a soft-margin SVM (not shown here), we could trade-off a “simpler” decision boundary against a few misclassified outliers.

Reproducing Kernel Hilbert Spaces

Formal Definition of Kernel Functions

A function $k : X \times X \rightarrow \mathbb{R}$ is a kernel if and only if it is symmetric and there exists a Hilbert space H and a map $\Phi : X \rightarrow H$ such that

$$k(\mathbf{x}, \mathbf{y}) = \langle \Phi(\mathbf{x}) \mid \Phi(\mathbf{y}) \rangle_H.$$

X does not have to be a vector space. It can also consist of nominal entries (e.g. {red, blue, green}), graphs or other “complicated” objects.

1st Direction

Given the mapping Φ and inner product $\langle \cdot \mid \cdot \rangle_H$ it is easy to construct the kernel k .

2nd Direction

Given a function k how can we verify that it is indeed a kernel? **This is the complicated part.**

Applying the kernel function on all pairs of data points in a set $\{\mathbf{y}_1, \dots, \mathbf{y}_n\}$ gives the **kernel matrix** (or Gram matrix) $\mathbf{K}$.

$$\mathbf{K} = \begin{pmatrix} k(\mathbf{y}_1, \mathbf{y}_1) & \cdots & k(\mathbf{y}_1, \mathbf{y}_n) \\ \vdots & \ddots & \vdots \\ k(\mathbf{y}_n, \mathbf{y}_1) & \cdots & k(\mathbf{y}_n, \mathbf{y}_n) \end{pmatrix}$$

Mercer's theorem [Mercer1909]:

The Gram matrix of a kernel function is **always positive semi-definite** (for all $\mathbf{x} \in \mathbb{R}^n$ follows $\mathbf{x}^\top \mathbf{K} \mathbf{x} \geq 0$) and symmetrical $\mathbf{K} = \mathbf{K}^\top$.

$$\begin{aligned} \text{Proof: } \mathbf{x}^\top \mathbf{K} \mathbf{x} &= \sum_i^n \sum_j^n \mathbf{x}_i \mathbf{x}_j \langle \Phi(\mathbf{y}_i) \mid \Phi(\mathbf{y}_j) \rangle \\ &= \langle \sum_{i=1}^n \mathbf{x}_i \Phi(\mathbf{y}_i) \mid \sum_{j=1}^n \mathbf{x}_j \Phi(\mathbf{y}_j) \rangle \\ &= \left\| \sum_{i=1}^n \mathbf{x}_i \Phi(\mathbf{y}_i) \right\|^2 \geq 0. \end{aligned}$$

Symmetry of $\mathbf{K}$ follows from the symmetry of k . $\square$

Kernels and Partial Evaluation

Consider any function $k : X \times X \rightarrow \mathbb{R}$. We can **partially evaluate the function** for an element $\mathbf{y} \in X$. This gives us $k(\mathbf{y}, \cdot)$, a function $X \rightarrow \mathbb{R}$. (Partial evaluation, and particularly the technique of **currying**, is commonly used in functional programming. Lisp, Haskell and ML all have it.) Here $\mathbf{y}$ is a fixed scalar and $(\cdot)$ is the remaining “free argument”. The notation might confuse at first, but it is a longstanding tradition.

Partial evaluation is a mapping to a (possibly infinite-dimensional) function space:

$$\Phi_k(\mathbf{y}) = k(\mathbf{y}, \cdot), \quad \Phi_k : X \rightarrow \mathbb{R}^X$$

We write $\mathbb{R}^X$ for the space of all function $X \rightarrow \mathbb{R}$. There are many inner products for function spaces. We select a very particular one:

$$\langle \Phi_k(\mathbf{x}) \mid \Phi_k(\mathbf{y}) \rangle = \langle k(\mathbf{x}, \cdot) \mid k(\mathbf{y}, \cdot) \rangle := \textcolor{red}{k(\mathbf{x}, \mathbf{y})}$$

But first we need a proper vector space. If the original set X has only a finite number of members, then there is only a finite number of functions $k(\mathbf{y}, \cdot)$. Allow all linear combinations of these “basis functions”.

$$G = \left\{ \sum_{i=1}^p \alpha_i k(\mathbf{y}_i, \cdot) \mid p \in \mathbb{N}_+, \alpha_i \in \mathbb{R}, \mathbf{y}_i \in X \right\} \quad \text{with } \mathbb{N}_+ \text{ the positive natural numbers}$$

Our “proto-inner product” on $\{\Phi_k(\mathbf{x}) \mid \mathbf{x} \in X\}$ automatically extends to all of G . For all $f, g \in G$:

$$\langle f \mid g \rangle_G = \langle \sum_i \alpha_i k(\mathbf{x}_i, \cdot) \mid \sum_j \beta_j k(\mathbf{y}_j, \cdot) \rangle_G = \sum_{i,j} \alpha_i \beta_j k(\mathbf{x}_i, \mathbf{y}_j)$$

Reproducing Kernel Hilbert Space

Necessary properties of k for $\langle \cdot | \cdot \rangle_G$ to hold the inner product axioms

- **Linearity** (addition and scalar multiplication) in the first argument of $\langle \cdot | \cdot \rangle_G$ comes by construction.
- For **symmetry** of $\langle \cdot | \cdot \rangle_G$ we need k symmetric as well (we don't consider conjugate symmetry).
- For **positive definiteness** ($\langle x | x \rangle \geq 0$ and $\langle x | x \rangle = 0$ iff $x = 0$) we need that property for k as well.

By consequence, every symmetric positive definite function is a valid kernel.

Let F a Hilbert space of functions $f : X \rightarrow \mathbb{R}$. Then F is called a **reproducing kernel Hilbert space (RKHS)** if there exists a kernel $k : X \times X \rightarrow \mathbb{R}$ such that

1. $\forall x \in X : k(x, \cdot) \in F$
2. $\forall f \in F : \langle f | k(x, \cdot) \rangle_F = f(x)$

The Hilbert space G we just constructed has the **reproducing property**: $\langle f | k(x, \cdot) \rangle_G = f(x)$

Proof: $\langle f | k(x, \cdot) \rangle_G = \sum_i \alpha_i \langle k(y_i, \cdot) | k(x, \cdot) \rangle = \sum_i \alpha_i k(y_i, x) = f(x)$

□

Furthermore, the RKHS is unique (up to isometric isomorphisms) for the kernel k [Aronszajn1950].

Combining Kernels

For the problem of designing kernels, see [Schölkopf2002] (Chapter 13) for general considerations and [Daemen2009] for a discussion in the context of clinical data.

Kernels can be transformed and combined. Let k_1, k_2 two valid kernels. For the following transformations, k' will still be a valid kernel. Remember that we only require the kernel function to be symmetric and positive definite.

Scalar multiplication $k'(x, y) = \alpha k_1(x, y)$ for $\alpha > 0$

Adding a constant $k'(x, y) = c + k_1(x, y)$ for $c \geq 0$

Addition of Kernels $k'(x, y) = k_1(x, y) + k_2(x, y)$

The Product of Kernels $k'(x, y) = k_1(x, y)k_2(x, y)$

Polynomials Let $p : \mathbb{R} \rightarrow \mathbb{R}$ a polynomial function with **only positive coefficients**. Then $k'(x, y) = p(k_1(x, y))$ is a valid kernel.

Exponential Function $k'(x, y) = \exp(k_1(x, y))$

See <https://www.cs.toronto.edu/~duvenaud/cookbook/> for a discussion of the “standard kernels” and their useful properties throughout transformations and combinations.

Summary of what you learned today

- **Linear classification** and training algorithms to compute linear classifiers
 - Perceptron
 - SVM
 - Training of the SVM as convex optimization
- **Nonlinear classification** by the use of **kernels**
- **Nonlinear Perceptron and SVM**
 - Use of the Lagrangian dual to “kernelize” the SVM
- The **Reproducing Kernel Hilbert Space** and criteria for functions to be valid kernels
- Ways to **combine kernels** with desired properties

- [Aronszajn1950] Nachman Aronszajn. “Theory of reproducing kernels”. In: *Transactions of the American mathematical society* 68.3 (1950), pp. 337–404.
- [Boser1992] Bernhard E Boser, Isabelle M Guyon, and Vladimir N Vapnik. “A training algorithm for optimal margin classifiers”. In: *Proceedings of the Fifth Annual Workshop on Computational Learning Theory*. 1992, pp. 144–152.
- [Cover1965] Thomas M Cover. “Geometrical and statistical properties of systems of linear inequalities with applications in pattern recognition”. In: *IEEE transactions on electronic computers* 3 (1965), pp. 326–334.
- [Daemen2009] Anneleen Daemen and Bart De Moor. “Development of a kernel function for clinical data”. In: *2009 Annual International Conference of the IEEE Engineering in Medicine and Biology Society*. IEEE. 2009, pp. 5913–5917.
- [Fisher1936] Ronald A Fisher. “The use of multiple measurements in taxonomic problems”. In: *Annals of Eugenics* 7.2 (1936), pp. 179–188.

- [Mercer1909] James Mercer. “Xvi. functions of positive and negative type, and their connection the theory of integral equations”. In: *Philosophical transactions of the royal society of London. Series A, containing papers of a mathematical or physical character* 209.441-458 (1909), pp. 415–446.
- [Rosenblatt1958] Frank Rosenblatt. “The perceptron: a probabilistic model for information storage and organization in the brain.”. In: *Psychological review* 65.6 (1958), p. 386.
- [Schölkopf2002] Bernhard Schölkopf and Alexander J Smola. *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press, 2002.